

Cloud Native Observability

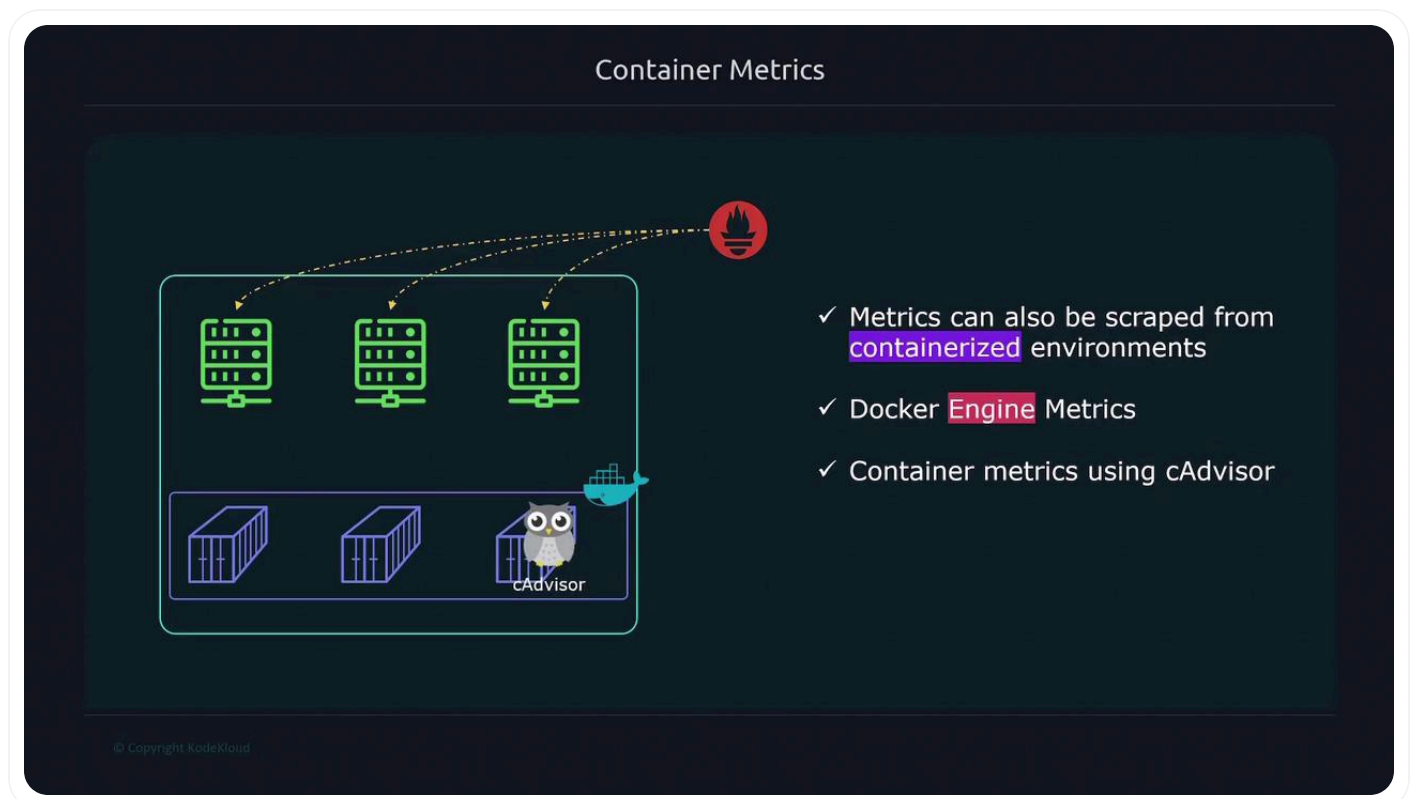
Prometheus Monitoring Containers

📄 Copy page



Learn to extend Prometheus monitoring from Linux hosts to containerized environments using cAdvisor for comprehensive system performance insights.

In this lesson, you will learn how to extend Prometheus monitoring from Linux hosts to containerized environments. By collecting metrics from both the Docker engine and individual containers using cAdvisor, you can gain comprehensive insights into your system's performance.



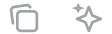
Enabling Docker Engine Metrics



>

This configuration exposes the Docker metrics endpoint and enables experimental features.

```
{  
  "metrics-addr": "127.0.0.1:9323",  
  "experimental": true  
}
```



2. Restart the Docker service to apply the changes:

```
$ sudo systemctl restart docker
```



3. Verify that the Docker metrics endpoint is accessible by running:

```
$ curl localhost:9323/metrics
```



Ensure that the experimental flag is enabled to properly expose Docker engine metrics.

Once you have verified the endpoint, update your Prometheus configuration to scrape Docker engine metrics. For example:

```
scrape_configs:  
  - job_name: "docker"  
    static_configs:  
      - targets: ["12.1.13.4:9323"]
```



Replace `12.1.13.4` with the actual IP address of your Docker host.



>

To deploy cAdvisor, follow these steps:

1. Create a Docker Compose file (e.g., `docker-compose.yml`) with the content below. This configuration is based on the official cAdvisor documentation:

```
version: '3.4'
services:
  cadvisor:
    image: gcr.io/cadvisor/cadvisor
    container_name: cadvisor
    privileged: true
    devices:
      - "/dev/kmsg:/dev/kmsg"
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker:/var/lib/docker:ro
      - /dev/disk:/dev/disk:ro
    ports:
      - 8080:8080
```



2. Start the cAdvisor service using Docker Compose:

```
$ docker-compose up -d
```



3. Verify that cAdvisor is collecting metrics by accessing its endpoint:

```
$ curl localhost:8080/metrics
```



>

```
- job_name: "cAdvisor"
  static_configs:
    - targets: ["12.1.13.4:8080"]
```

Replace `12.1.13.4` with the IP address where cAdvisor is running.

Docker Metrics vs cAdvisor Metrics



Docker Engine metrics

- ✓ How much cpu does **docker** use
- ✓ **Total** number of failed image builds
- ✓ Time to process container **actions**
- ✓ No metrics **specific** to a container



cAdvisor metrics

- ✓ How much cpu/mem does **each** container use
- ✓ Number of **processes** running inside a container
- ✓ **Container** uptime
- ✓ Metrics on a **per container** basis

© Copyright KodeKloud

Docker vs. cAdvisor Metrics

Understanding the difference between Docker engine metrics and cAdvisor metrics is important for effective monitoring:

Docker Engine Metrics:

These metrics provide information about the overall Docker engine performance, such as:

Overall CPU usage of the Docker engine.



>

cAdvisor Metrics:

cAdvisor provides granular metrics for each container including:

CPU and memory usage.

Number of processes running inside a container.

Container uptime.

Use cAdvisor metrics when you require detailed, container-specific performance data.

Use Docker engine metrics for a holistic view of your host's performance and cAdvisor for in-depth analysis of each container. Adjust your Prometheus configuration accordingly to ensure efficient monitoring.



Watch Video

[Learn more >](#)

Was this page helpful?

Yes

No

Prometheus Metrics

[< Previous](#)

Prometheus Monitoring Kubernetes

[Next >](#)



>
